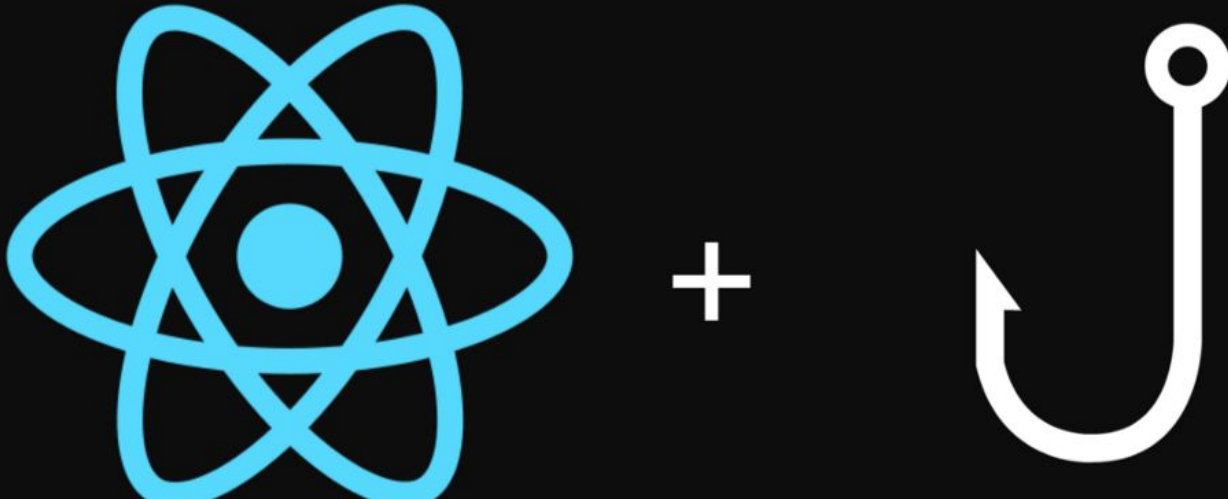


Identifica qué son los Hooks

Clase 3



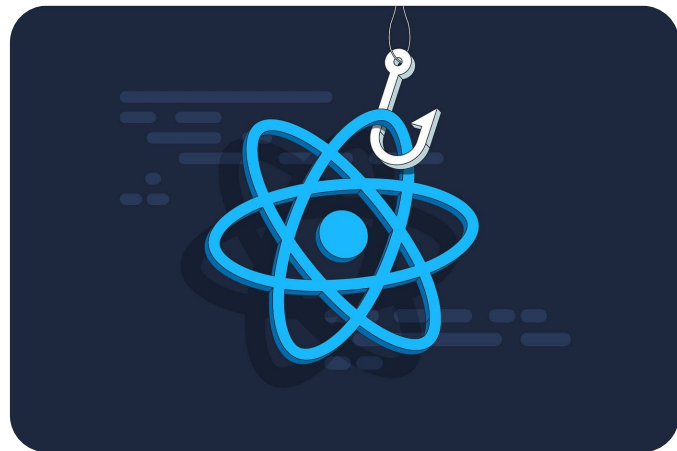


¿Qué son los **Hooks**?

Los Hooks son **funciones introducidas en React 16.8** que permiten **engancharse al estado** y otras características de React dentro de los componentes funcionales.

Antes de los Hooks, los componentes de clase manejaban el estado y los ciclos de vida, mientras que los componentes funcionales eran más limitados.

Los Hooks ofrecen una **solución elegante** a las limitaciones anteriores, permitiendo el uso del estado y otras características de React en todos los componentes, sin necesidad de convertirlos en clases.

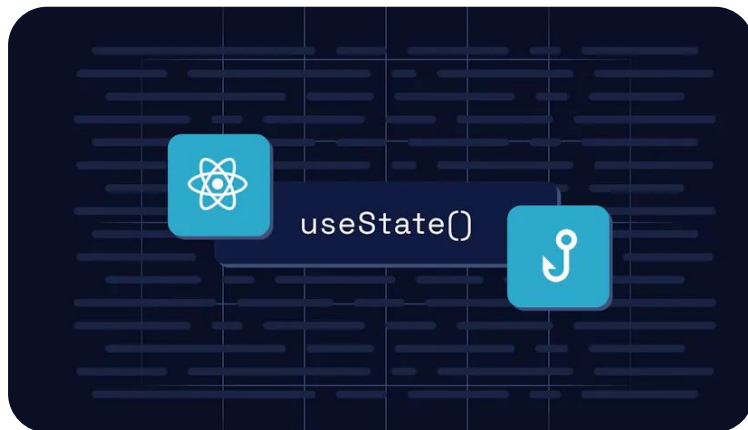




Principales Hooks

Se mencionan dos de los Hooks más utilizados en React: `useState` y `useEffect`.

- **`useState`:** Permite agregar estado a un componente funcional.
- **`useEffect`:** Permite ejecutar efectos secundarios en un componente funcional, como realizar solicitudes de datos, suscripciones, etc.

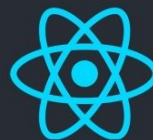




Sintaxis y Uso de **useState**:

- **useState** es una función que devuelve un array con dos elementos: el estado actual y una función para actualizar dicho estado.
- Se puede inicializar el estado proporcionando un valor dentro de useState.

```
const [movies, setMovies] = useState([]);
```

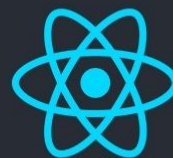


useState
hook in Action



Uso de **useEffect**:

- **useEffect** permite ejecutar una función de efecto secundario en cada renderizado o cuando ciertas dependencias cambian.
- Se puede utilizar para realizar operaciones adicionales más allá del renderizado, como solicitudes de datos.
- Se puede controlar cuándo se ejecuta el efecto especificando dependencias.



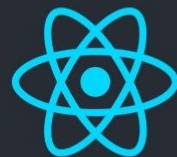
The
useEffect Hook

```
useEffect(() => {
  const fetchMovies = async () => {
    // Simulación de solicitud de datos
    const response = await fetchMovies();
    setMovies(response);
  };
  fetchMovies();
}, []); // Ejecuta el efecto solo una vez al montar el componente
```



Dependencias en **useEffect**:

- Las dependencias en `useEffect` permiten controlar cuándo se ejecuta el efecto en respuesta a cambios específicos en el estado o las props.
- Si una variable externa se utiliza en el efecto, React sugiere incluirla como dependencia para garantizar que el efecto se ejecute nuevamente si esa variable cambia.
- Las dependencias se pasan como un segundo argumento a `useEffect`.



The
useEffect Hook

```
useEffect(() => {  
  // Código del efecto  
}, [dependency]); // Ejecutar el efecto cuando  
  'dependency' cambie
```

